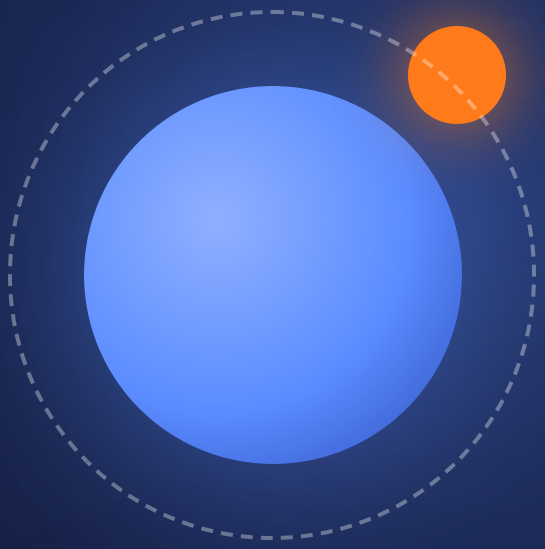


EDICIÓN 2026

Lua para FiveM

De cero a tu primer script.
Con ejemplos reales en cada framework.

ESX Legacy · QBCore · Qbox · ND Core · vRP · Standalone + ox_lib

JVL MediaCode
Jose M. Valenzuela

Lua para FiveM — Edición 2026

© 2026 JVL MediaCode · Jose M. Valenzuela. Todos los derechos reservados.

Este libro se vende para uso personal. No está permitido revenderlo, compartirlo ni publicarlo, completo o en partes, sin permiso escrito del autor. El código de los ejemplos sí puedes usarlo libremente en tus propios servidores y scripts.

FiveM, Cfx.re, ESX, QBCore, Qbox, ND Core, vRP, ox_lib y ox_inventory son marcas o proyectos de sus respectivos autores. Este libro no está afiliado a ninguno de ellos.

Las APIs de los frameworks cambian con el tiempo. Los ejemplos se revisaron contra la documentación oficial de octubre de 2026.

contacto@jvlmediacode.com · jvlmediacode.com

Contenido

Antes de empezar

LUA ESENCIAL

1. Lua en FiveM
2. Variables, tipos y vectores
3. Operadores y texto
4. Condiciones y ciclos
5. Funciones
6. Tablas
7. Compartir código y POO ligera
8. Errores y depuración

FIVEM POR DENTRO

9. Anatomía de un resource
10. Cliente vs servidor
11. Eventos
12. Hilos y Wait
13. Natives: hablar con el juego
14. Callbacks, exports y state bags
15. Base de datos con oxmysql
16. Interfaces NUI (lo básico)
17. Seguridad: nunca confíes en el cliente
18. Rendimiento

FRAMEWORKS

19. Qué es un framework y cuál elegir

20. ESX Legacy

21. QBCore

22. Qbox

23. ND Core

24. vRP

25. Standalone con ox_lib y ox_inventory

26. La misma tarea en todos los frameworks

27. Un script para todos los frameworks

PROYECTO FINAL

28. Proyecto: trabajo de repartidor

PARA TENER A LA MANO

A. Errores comunes y su solución

B. Chuleta rápida

Antes de empezar

Este libro tiene un solo objetivo: que al terminarlo puedas **escribir, leer y arreglar scripts de FiveM** sin copiar y pegar a ciegas.

No es un manual genérico de Lua. Todo lo que no sirve en FiveM se quedó fuera: no vas a leer archivos de texto, no vas a instalar LuaRocks y no vas a hacer calculadoras de consola. Cada ejemplo es algo que podrías poner en un servidor de verdad: jugadores, dinero, items, trabajos, comandos, marcadores y eventos.

Cómo está organizado

Parte	Qué aprendes
1. Lua esencial	El lenguaje, explicado con ejemplos de FiveM desde la primera línea.
2. FiveM por dentro	Resources, cliente y servidor, eventos, hilos, natives, base de datos, seguridad y rendimiento.
3. Frameworks	ESX Legacy, QBCore, Qbox, ND Core, vRP y standalone con ox_lib, con la **misma tarea** resuelta en cada uno.
4. Proyecto final	Un trabajo completo, de principio a fin, que funciona en cualquier framework.

Si ya programas en otro lenguaje, puedes leer la Parte 1 rápido. Si nunca has programado, léela con calma y escribe cada ejemplo tú mismo: **escribirlo enseña más que leerlo**.

Lo que necesitas

- **Un servidor local de FiveM** (FXServer con txAdmin) para probar sin miedo a romper nada. No pruebes en el servidor donde juega tu comunidad.
- **Visual Studio Code** con la extensión **Lua** (de *sumneko*). Te marca errores mientras escribes.
- **La consola**: `F8` en el juego (cliente) y la ventana del servidor o txAdmin (servidor). Ahí aparecen tus `print` y tus errores.

Cómo leer los recuadros

> 💡 **Tip**
>
> Atajos y buenas prácticas que te ahorran tiempo.

⚠️ **Trampa real**

Errores que pasan de verdad en servidores en producción, y cómo evitarlos.

🔒 **Seguridad**

Lo que separa un script sano de uno que los tramposos explotan en cinco minutos.

✅ **Resumen**

Lo más importante del capítulo, en pocas líneas.

1. Lua en FiveM

Lua es un lenguaje pequeño, rápido y fácil de leer, creado en Brasil en 1993. Su nombre significa "luna" en portugués. Está pensado para meterse dentro de otros programas, y por eso lo usan juegos como World of Warcraft, Roblox (en una versión propia) y, lo que nos importa, **FiveM**.

FiveM usa su propia versión, **CfxLua**: es **Lua 5.4** con algunos extras para el juego (vectores, hashes y funciones del motor). Todo lo que aprendas de Lua 5.4 te sirve, y en este libro te señalamos los extras cuando aparezcan.

Dónde corre tu código

Esta es la idea más importante de todo el libro, así que va primero: **en FiveM tu código corre en dos lugares distintos**.

	Cliente (`client.lua`)	Servidor (`server.lua`)
Dónde corre	En la PC de **cada jugador**	En **tu servidor**, una sola vez
Qué controla	Lo que el jugador ve: marcadores, teclas, menús, efectos	Lo que importa: dinero, items, base de datos, permisos
¿Puede tocarlo un tramposo?	**Sí.** Lo puede leer y modificar	No. Solo tú tienes acceso
Consola	`F8` en el juego	Ventana del servidor / txAdmin

Imagina un banco: el **cliente** es la ventanilla, donde el jugador pide cosas, y el **servidor** es la bóveda, que decide si se las da. Nunca dejes que la ventanilla abra la bóveda sola.

> 🔒 **Seguridad**
>
> Todo lo que viene del cliente puede ser falso. Si el cliente dice "págame $5,000", el servidor **no** debe creerle: tiene que comprobar por su cuenta si el jugador se lo ganó. Lo vemos a fondo en el capítulo de seguridad.

Tu primer script

Un script de FiveM se llama **resource**: una carpeta con un archivo `fxmanifest.lua` que le dice a FiveM qué archivos cargar. Crea esta estructura dentro de `resources/` de tu servidor:

```
resources/
└── [mis_scripts]/
    └── hola_mundo/
        ├── fxmanifest.lua
        ├── client.lua
        └── server.lua
```

Las carpetas con corchetes, como `[mis_scripts]`, son solo para ordenar: FiveM busca resources dentro de ellas.

fxmanifest.lua: la ficha del resource.

```
fx_version 'cerulean'
game 'gta5'
lua54 'yes'

author 'Tu nombre'
description 'Mi primer script'
version '1.0.0'

client_script 'client.lua'
server_script 'server.lua'
```

server.lua: corre en el servidor.

```
print('Hola desde el servidor')

-- Se ejecuta cada vez que un jugador termina de conectarse
AddEventHandler('playerJoining', function()
    local jugador = source
    print(('Entró %s (id %d)'):format(GetPlayerName(jugador), jugador))
end)
```

client.lua: corre en la PC del jugador.

```
print('Hola desde el cliente')

RegisterCommand('hola', function()
    local coords = GetEntityCoords(PlayerPedId())
    print(('Estás en x=%.1f y=%.1f z=%.1f'):format(coords.x, coords.y, coords.z))
end, false)
```

Agrega `ensure hola_mundo` a tu `server.cfg` (o escribe `ensure hola_mundo` en la consola del servidor) y entra al juego. En la consola del servidor verás "*Hola desde el servidor*" y en `F8` verás "*Hola desde el cliente*". Escribe `/hola` en el chat y aparecerán tus coordenadas.

> 💡 **Tip**
>
> Cada vez que cambies el código, escribe `restart hola_mundo` en la consola del servidor. No hace falta reiniciar el servidor completo.

Qué acabas de usar

En diez líneas ya usaste casi todo lo que aprenderás en este libro:

- **Variables** (`local jugador`, `local coords`), en el capítulo 2.
- **Funciones** (`function() ... end`), en el capítulo 5.
- **Eventos** (`AddEventHandler`), en la Parte 2.
- **Natives**: funciones del juego como `GetEntityCoords`, `PlayerPedId` y `GetPlayerName`, también en la Parte 2.
- **Vectores**: `coords` es un `vector3` con `.x`, `.y` y `.z`, un extra de CfxLua.

> ⚠️ **Trampa real**
>
> `PlayerPedId()` solo existe en el **cliente**. Si lo usas en `server.lua` verás "*attempt to call a nil value (global 'PlayerPedId')*". Muchas natives son solo de cliente o solo de servidor: la documentación oficial (`docs.fivem.net/natives`) indica de qué lado funciona cada una.

> ✅ **Resumen**
>
> - FiveM usa Lua 5.4 con extras (CfxLua).
> - El **cliente** corre en la PC de cada jugador y no es confiable; el **servidor** decide lo importante.
> - Un resource es una carpeta con `fxmanifest.lua`, y se carga con `ensure` y se recarga con `restart`.

17. Seguridad: nunca confíes en el cliente

Este es el capítulo más importante para un servidor en producción. Los menús de trampas (*mod menus*) permiten a un jugador **leer todo tu código de cliente** y **disparar cualquier evento de red con los argumentos que quiera**. No es una posibilidad remota: en un servidor con algo de público pasa cada semana.

La buena noticia es que defenderse no requiere un anticheat caro, sino escribir los eventos del servidor con cuatro hábitos.

El ataque clásico

Tu script tiene esto:

```lua
-- server.lua ❌
RegisterNetEvent('pesca:vender', function(cantidad, precio)
    local xPlayer = ESX.GetPlayerFromId(source)
    xPlayer.addAccountMoney('money', cantidad * precio)
end)
```

El tramposo abre su menú, busca eventos (los ve en tu `client.lua`) y ejecuta:

```lua
TriggerServerEvent('pesca:vender', 1, 99999999)
```

Cien millones al instante, sin haber pescado nada. Y lo repite en un ciclo.

Hábito 1: el servidor pone los números

El cliente nunca manda precios, pagos ni recompensas. Solo dice **qué** quiere hacer; el servidor saca los valores de su config.

```lua
-- config.lua
Config.Precios = { pescado = 25, pescado_raro = 120 }
```

```lua
-- server.lua ✅
RegisterNetEvent('pesca:vender', function(item)
    local src = source
    local precio = Config.Precios[item]
    if not precio then return end -- item que no se vende: ignorar
```

```
    -- el servidor cuenta cuántos tiene DE VERDAD en el inventario
    local cantidad = contarItem(src, item)
    if cantidad <= 0 then return end

    quitarItem(src, item, cantidad)
    darDinero(src, cantidad * precio)
end)
```

Hábito 2: valida quién y dónde

Antes de hacer cualquier cosa, comprueba que la acción **tiene sentido** para ese jugador en ese momento:

```
local PUNTO_VENTA = vector3(-1845.9, -1196.5, 14.3)

RegisterNetEvent('pesca:vender', function(item)
    local src = source

    -- ¿Tiene el trabajo correcto?
    if obtenerTrabajo(src) ~= 'pescador' then return end

    -- ¿Está realmente en el punto de venta? (el servidor lo mide, no le cree al cliente)
    local coords = GetEntityCoords(GetPlayerPed(src))
    if #(coords - PUNTO_VENTA) > 5.0 then
        sospechoso(src, ('vender a %.0f m del punto'):format(#(coords - PUNTO_VENTA)))
        return
    end

    -- ... vender
end)
```

Hábito 3: límite de velocidad (*cooldown*)

Aunque todo sea legítimo, nadie vende 50 veces por segundo. Un cooldown por jugador corta los ciclos automáticos:

```
local ultimaAccion = {}

local function enCooldown(src, segundos)
    local ahora = os.time()
    if ultimaAccion[src] and ahora - ultimaAccion[src] < segundos then
        return true
    end
    ultimaAccion[src] = ahora
    return false
```

```lua
end

RegisterNetEvent('pesca:vender', function(item)
    local src = source
    if enCooldown(src, 3) then return end
    -- ...
end)

AddEventHandler('playerDropped', function()
    ultimaAccion[source] = nil -- no acumular basura
end)
```

Hábito 4: el estado vive en el servidor

Si una acción tiene pasos (tomar un paquete → llevarlo → entregarlo), el servidor debe **recordar en qué paso va cada jugador**. Si no, el tramposo se salta directo al último:

```lua
local paqueteEnMano = {}

RegisterNetEvent('reparto:tomarPaquete', function()
    local src = source
    -- validar distancia al almacén...
    paqueteEnMano[src] = { destino = math.random(#Config.Destinos), desde = os.time() }
end)

RegisterNetEvent('reparto:entregar', function()
    local src = source
    local paquete = paqueteEnMano[src]
    if not paquete then return end              -- no tomó ningún paquete

    if os.time() - paquete.desde < 20 then      -- imposible llegar tan rápido
        sospechoso(src, 'entrega demasiado rápida')
        return
    end

    -- validar distancia a Config.Destinos[paquete.destino]...
    paqueteEnMano[src] = nil
    darDinero(src, Config.Pago)
end)
```

Registrar a los sospechosos

No hace falta banear automáticamente (puedes equivocarte), pero sí **dejar registro** para que un admin revise:

```lua
local WEBHOOK = GetConvar('reparto_webhook', '') -- en server.cfg con "set", nunca en el config

function sospechoso(src, motivo)
    local nombre = GetPlayerName(src) or 'desconocido'
    local licencia = GetPlayerIdentifierByType(src, 'license') or '?'
    print(('[reparto][SOSPECHOSO] %s (%s): %s'):format(nombre, licencia, motivo))

    if WEBHOOK == '' then return end
    PerformHttpRequest(WEBHOOK, function() end, 'POST', json.encode({
        content = ('⚠️ **%s** (`%s`): %s'):format(nombre, licencia, motivo),
    }), { ['Content-Type'] = 'application/json' })
end
```

Lista de revisión para cada evento de red

Antes de dar por terminado cualquier `RegisterNetEvent` del servidor, repasa:

☐ `local src = source` en la primera línea.

☐ El jugador existe (el framework devolvió su objeto).

☐ Ningún precio, pago o cantidad viene del cliente sin validar.

☐ Tiene el trabajo, rango o item necesario (comprobado en el servidor).

☐ Está cerca del lugar donde ocurre la acción (medido por el servidor).

☐ Hay un cooldown si la acción se puede repetir.

☐ Si la acción tiene pasos, el servidor recuerda en qué paso va.

☐ Los textos que escribió el jugador se validan (largo y caracteres).

☐ Las consultas a la base de datos usan `?`.

> 🔒 **Seguridad**
>
> Los secretos (webhooks, tokens, contraseñas de APIs) van en `server.cfg` con `set` y se leen con `GetConvar` solo en el servidor. Nunca en `config.lua`, nunca en archivos de cliente y nunca subidos a GitHub.

Resumen

- El tramposo puede leer tu cliente y disparar cualquier evento con cualquier argumento.
- **Cuatro hábitos:** el servidor pone los números; valida quién y dónde; cooldown; el estado vive en el servidor.
- Registra lo sospechoso en vez de banear a ciegas.
- Pasa la lista de revisión a cada evento de red.

Fin de la muestra

Esto fue la bienvenida y dos de los **28 capítulos** del libro. El libro completo incluye todo Lua desde cero, FiveM por dentro, ESX, QBCore, Qbox, ND, vRP y standalone, y un proyecto final probado en un servidor real.

Consíguelo en Zeltrade. El código del proyecto final es gratis en **github.com/jvalenzmedia/jvl_reparto**.